

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Repository Link: https://github.com/Atharsh2910/Deep_Learning_Lab

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	ATHARSH S U	
Register Number	24011101008	
Roll Number	24110049	
Class	AI-DS 'A'	

1 Objective

To design, implement, and systematically optimize a Convolutional Neural Network (CNN) for multi-class image classification on the CIFAR-10 dataset. The experiment explores hyperparameter tuning strategies, regularization techniques (Batch Normalization, Dropout, L2), data augmentation, and adaptive learning rate scheduling to achieve improved generalization beyond a baseline CNN.

2 Learning Outcomes

Students will be able to:

1. Implement data augmentation using `ImageDataGenerator` to reduce overfitting.
2. Apply Batch Normalization and Dropout as regularization strategies.
3. Design a deeper CNN with progressive filter expansion ($32 \rightarrow 64 \rightarrow 128$).
4. Use `GlobalAveragePooling2D` as a parameter-efficient alternative to `Flatten`.
5. Employ adaptive callbacks (`EarlyStopping`, `ReduceLROnPlateau`, `ModelCheckpoint`).
6. Compare ReLU vs Sigmoid activations quantitatively.
7. Evaluate models using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.
8. Interpret and compare baseline vs optimized model performance.

3 Background Theory

3.1 Convolutional Neural Network (CNN)

A CNN consists of stacked convolution, activation, pooling, and fully connected layers. The convolution operation is:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) \quad (1)$$

where X is the input, K is the kernel, and Y is the feature map. The output spatial size is:

$$\text{Output Size} = \frac{N - F + 2P}{S} + 1 \quad (2)$$

where N = input size, F = filter size, P = padding, S = stride.

3.2 Batch Normalization

Batch Normalization (BN) normalizes the activations of a layer across a mini-batch:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (3)$$

It reduces internal covariate shift, speeds up convergence, and acts as a mild regularizer.

3.3 Dropout

Dropout randomly sets a fraction p of activations to zero during training, preventing co-adaptation of neurons and reducing overfitting.

3.4 L2 Regularization

L2 (weight decay) adds a penalty $\lambda \sum w_i^2$ to the loss function, constraining the magnitude of weights.

3.5 Data Augmentation

Augmentation artificially expands the training set by applying random transformations (rotation, flip, shift, zoom), helping the model generalize better.

3.6 GlobalAveragePooling2D

GlobalAveragePooling2D (GAP) computes the spatial mean of each feature map, producing a compact vector. It drastically reduces the number of parameters compared to a Flatten + Dense block.

3.7 Adaptive Learning Rate Scheduling

ReduceLROnPlateau monitors a metric (e.g., validation loss) and reduces the learning rate by a factor when improvement stalls, allowing finer optimization. **EarlyStopping** halts training when validation performance ceases to improve, restoring the best weights.

3.8 Activation Function Comparison: ReLU vs Sigmoid

- **ReLU:** $f(x) = \max(0, x)$ – computationally efficient, avoids vanishing gradient for positive activations.
- **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$ – saturates at extremes, prone to vanishing gradients in deep networks.

4 Dataset: CIFAR-10

- **Training Images:** 50,000 ($32 \times 32 \times 3$ RGB)
- **Testing Images:** 10,000 ($32 \times 32 \times 3$ RGB)
- **Classes:** 10 (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
- **Preprocessing:** Pixel values normalized to $[0, 1]$ by dividing by 255. Labels one-hot encoded.

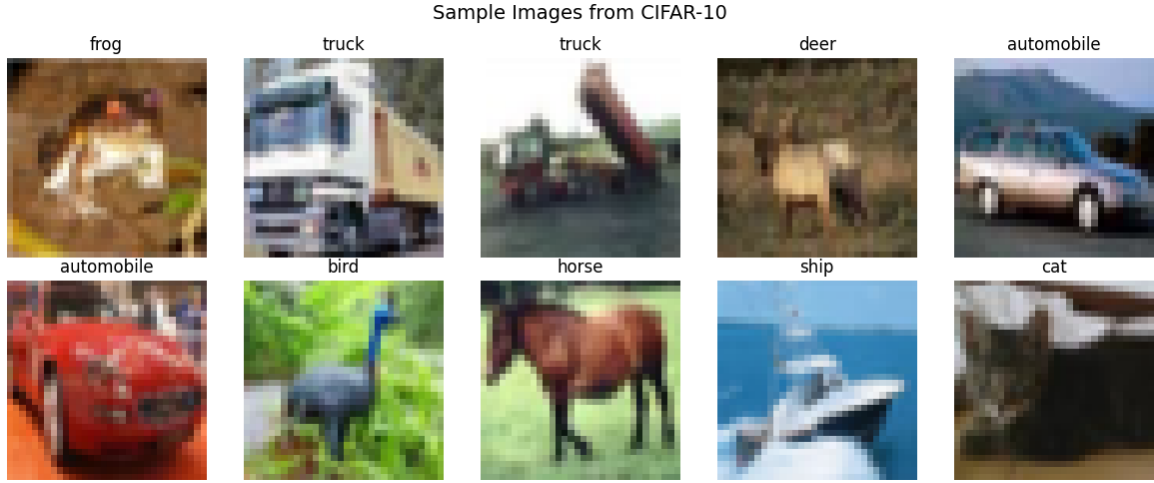


Figure 1: Sample images from the CIFAR-10 dataset showing all 10 classes at 32×32 resolution. The dataset exhibits significant intra-class variation and inter-class visual similarity (e.g., cat vs dog), making it a challenging benchmark.



Figure 2: Class distribution across the CIFAR-10 training set. Each class contains exactly 5,000 images, confirming a perfectly balanced dataset. No re-sampling or class weighting is required.

5 Analytical Exercises

5.1 Output Size Calculation

For a 64×64 input image with a 5×5 kernel, stride $S = 2$, and padding $P = 2$:

$$\text{Output Size} = \frac{64 - 5 + 2 \times 2}{2} + 1 = \frac{63}{2} + 1 = 32 \times 32 \quad (4)$$

5.2 Trainable Parameter Count

For a Conv2D layer with 64 filters of size 3×3 applied to an RGB (3-channel) input:

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1,792 \quad (5)$$

6 Activation Function Comparison: ReLU vs Sigmoid

Both ReLU and Sigmoid activations were evaluated on a simple 2-Conv CNN trained for 5 epochs on CIFAR-10:

Activation	Test Accuracy	Training Time
ReLU	57.05%	9.1 s
Sigmoid	33.29%	7.6 s

Table 1: ReLU vs Sigmoid – 5-epoch comparison on CIFAR-10. ReLU achieves significantly higher accuracy due to its linear gradient flow for positive activations, avoiding saturation.

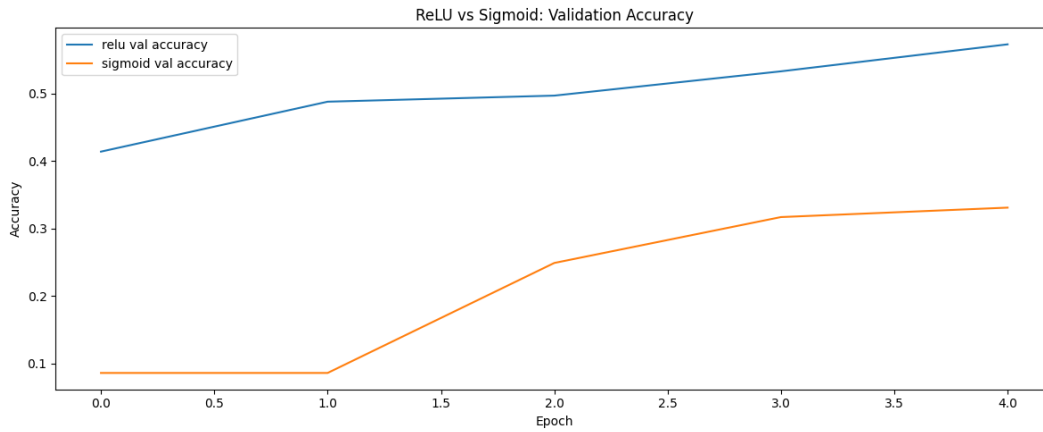


Figure 3: Validation accuracy over 5 epochs for ReLU (blue) and Sigmoid (orange) activations. ReLU converges faster and achieves higher accuracy. Sigmoid suffers from vanishing gradients, especially in early epochs (near-random 8.6% in epoch 1–2).

7 Feature Map Visualization

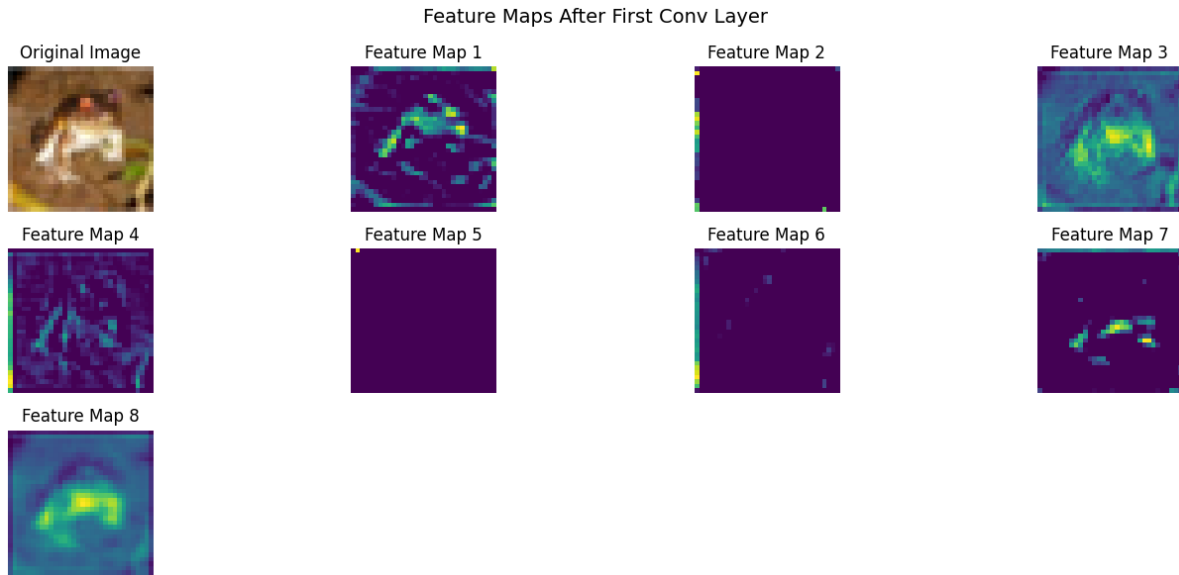


Figure 4: Feature maps extracted from the first convolutional layer of the baseline CNN for a sample frog image. Feature Map 1 highlights edge contours; Feature Map 3 captures circular blob-like regions (head); Feature Map 8 detects large smooth gradient regions. Darker maps indicate low activation for that filter’s specific pattern.

8 Optimized CNN Architecture

8.1 Architecture Design

The optimized model replaces the flat baseline architecture with a **three-block convolutional structure** using progressively increasing filter counts ($32 \rightarrow 64 \rightarrow 128$), Batch Normalization, Dropout, and GlobalAveragePooling2D:

Listing 1: Optimized CNN architecture (conv_block function + head)

```
def conv_block(x, filters, l2_reg=1e-4):
    x = layers.Conv2D(filters, (3,3), padding='same',
                      kernel_regularizer=regularizers.l2(l2_reg))(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Conv2D(filters, (3,3), padding='same',
                      kernel_regularizer=regularizers.l2(l2_reg))(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.MaxPooling2D((2, 2))(x)
    x = layers.Dropout(0.25)(x)
    return x

inputs = layers.Input(shape=(32, 32, 3))
x = conv_block(inputs, 32)
x = conv_block(x, 64)
x = conv_block(x, 128)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation='relu',
```

```

        kernel_regularizer=regularizers.l2(1e-4))(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(10, activation='softmax')(x)
tuned_model = models.Model(inputs, outputs, name='tuned_cifar10_cnn')

```

8.2 Model Summary

Layer	Output Shape	Params
Input	(None, 32, 32, 3)	0
Conv2D (32 filters)	(None, 32, 32, 32)	896
BatchNormalization	(None, 32, 32, 32)	128
Activation (ReLU)	(None, 32, 32, 32)	0
Conv2D (32 filters)	(None, 32, 32, 32)	9,248
BatchNormalization	(None, 32, 32, 32)	128
Activation (ReLU)	(None, 32, 32, 32)	0
MaxPooling2D	(None, 16, 16, 32)	0
Dropout (0.25)	(None, 16, 16, 32)	0
Conv2D (64 filters)	(None, 16, 16, 64)	18,496
BatchNormalization	(None, 16, 16, 64)	256
Activation (ReLU)	(None, 16, 16, 64)	0
Conv2D (64 filters)	(None, 16, 16, 64)	36,928
BatchNormalization	(None, 16, 16, 64)	256
Activation (ReLU)	(None, 16, 16, 64)	0
MaxPooling2D	(None, 8, 8, 64)	0
Dropout (0.25)	(None, 8, 8, 64)	0
Conv2D (128 filters)	(None, 8, 8, 128)	73,856
BatchNormalization	(None, 8, 8, 128)	512
Activation (ReLU)	(None, 8, 8, 128)	0
Conv2D (128 filters)	(None, 8, 8, 128)	147,584
BatchNormalization	(None, 8, 8, 128)	512
Activation (ReLU)	(None, 8, 8, 128)	0
MaxPooling2D	(None, 4, 4, 128)	0
Dropout (0.25)	(None, 4, 4, 128)	0
GlobalAveragePooling2D	(None, 128)	0
Dense (256 units)	(None, 256)	33,024
BatchNormalization	(None, 256)	1,024
Dropout (0.50)	(None, 256)	0
Dense (10 units, Softmax)	(None, 10)	2,570

9 Training Configuration and Callbacks

9.1 Data Augmentation

Listing 2: ImageDataGenerator configuration for on-the-fly data augmentation

```
datagen = ImageDataGenerator(  
    rotation_range=15,  
    width_shift_range=0.1,  
    height_shift_range=0.1,  
    zoom_range=0.1,  
    horizontal_flip=True,  
    fill_mode='nearest'  
)  
datagen.fit(x_train_norm)
```

9.2 Callbacks

Listing 3: Training callbacks for adaptive optimization and checkpointing

```
callbacks_list = [  
    callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,  
                                patience=4, min_lr=1e-6, verbose=1),  
    callbacks.EarlyStopping(monitor='val_loss', patience=10,  
                             restore_best_weights=True, verbose=1),  
    callbacks.ModelCheckpoint('best_cifar10_model.keras',  
                              monitor='val_accuracy',  
                              save_best_only=True, verbose=1)  
]
```

9.3 Training Parameters

Parameter	Value
Optimizer	Adam
Initial Learning Rate	0.001
Batch Size	64
Max Epochs	60 (with EarlyStopping)
LR Reduce Factor	0.5
LR Patience	4 epochs
Early Stop Patience	10 epochs
Loss Function	Categorical Cross-Entropy

Table 2: Hyperparameters used for training the optimized CNN.

10 Training Results and Curves

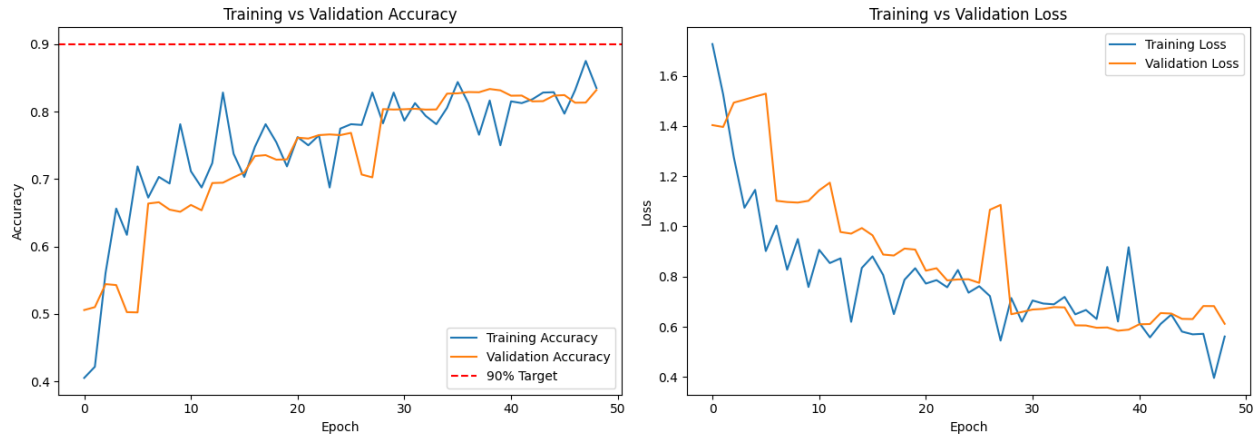


Figure 5: Training vs Validation Accuracy (left) and Loss (right) for the optimized CNN over ~ 49 epochs. The model steadily improves, reaching $\sim 83\%$ validation accuracy. The red dashed line marks the 90% target. The loss curves converge without severe overfitting – a result of BN + Dropout + L2 regularization combined with data augmentation. The LR was halved at epoch 6 when validation loss stalled.

11 Evaluation Metrics

11.1 Optimized CNN – Test Set Performance

Metric	Value
Accuracy	0.8334 (83.34%)
Precision (macro)	0.8385
Recall (macro)	0.8334
F1-score (macro)	0.8307

Table 3: Test-set evaluation metrics for the optimized CNN on CIFAR-10.

11.2 Per-Class Classification Report (Optimized CNN)

Class	Precision	Recall	F1-score	Support
Airplane	0.83	0.86	0.85	1000
Automobile	0.87	0.97	0.92	1000
Bird	0.88	0.67	0.76	1000
Cat	0.79	0.63	0.70	1000
Deer	0.87	0.76	0.81	1000
Dog	0.79	0.77	0.78	1000
Frog	0.70	0.96	0.81	1000
Horse	0.90	0.87	0.88	1000
Ship	0.90	0.91	0.91	1000
Truck	0.86	0.93	0.90	1000
Macro Avg	0.84	0.83	0.83	10000

Table 4: Per-class precision, recall, and F1-score for the optimized CNN. Automobile achieves the highest F1 (0.92) while Cat remains the hardest class (F1: 0.70) due to visual similarity with Dog.

11.3 Confusion Matrix – Optimized CNN

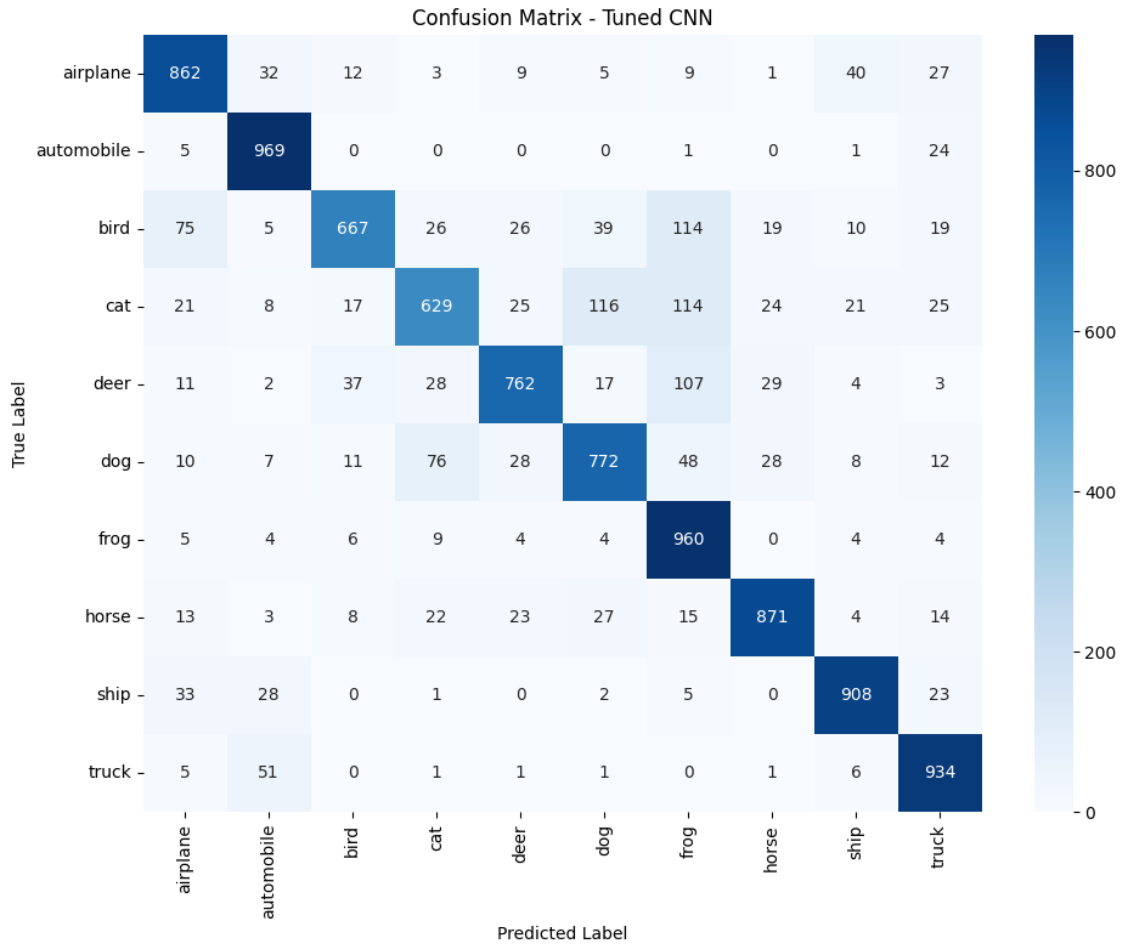


Figure 6: Confusion matrix for the optimized CNN on the CIFAR-10 test set. The diagonal dominance confirms strong per-class performance. Notable confusions: Bird↔Frog (114 misclassifications), Cat↔Dog (116+114), and Deer↔Frog (107). Automobile achieves near-perfect recall (969/1000).

12 Comparison: Baseline CNN vs Optimized CNN

The baseline CNN (`d1-lab3.ipynb`) is a simple 2-block model without regularization or augmentation, trained for 20 epochs with batch size 32. The optimized CNN (`d1-lab3-opt.ipynb`) applies BatchNorm, Dropout, L2, data augmentation, and adaptive LR scheduling.

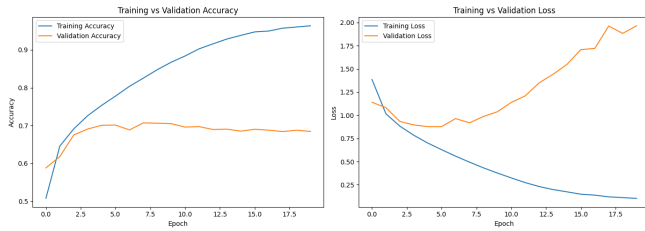
12.1 Hyperparameter and Architecture Comparison

Aspect	Baseline CNN	Optimized CNN
Architecture	Conv(32)→Pool → Conv(64) → Pool → Flatten → Dense(128) → Softmax	3×[Conv(F)→BN→ReLU→ → GAP → Dense(256)→BN→Drop → Softmax
Conv Filters	32, 64	32, 32, 64, 64, 128, 128
Batch Normalization	None	After every Conv and Dense
Dropout	None	0.25 (conv), 0.50 (dense)
L2 Regularization	None	$\lambda = 10^{-4}$
Data Augmentation	None	Rotation, flip, shift, zoom
Optimizer	Adam (default lr)	Adam (lr = 10^{-3})
Batch Size	32	64
Epochs	20 (fixed)	Up to 60 with EarlyStopping
LR Scheduling	None	ReduceLROnPlateau (factor 0.5, patience 4)
Early Stopping	None	patience = 10, restore best weights
Pooling Type	MaxPooling2D	MaxPooling2D
Flatten Strategy	Flatten → Dense(128)	GlobalAveragePooling2D → Dense(256)

12.2 Performance Metric Comparison

Metric	Baseline CNN	Optimized CNN	Improvement
Test Accuracy	68.43%	83.34%	+14.91%
Precision (macro)	68.64%	83.85%	+15.21%
Recall (macro)	68.43%	83.34%	+14.91%
F1-score (macro)	68.44%	83.07%	+14.63%
Best Airplane F1	0.73	0.85	+0.12
Best Cat F1	0.48	0.70	+0.22
Best Automobile F1	0.79	0.92	+0.13
Overfitting	High	Mild	Significant reduction
Epochs to converge	20 (fixed, overfits)	~49 (early stopped at best)	Adaptive

12.3 Training Curve Comparison



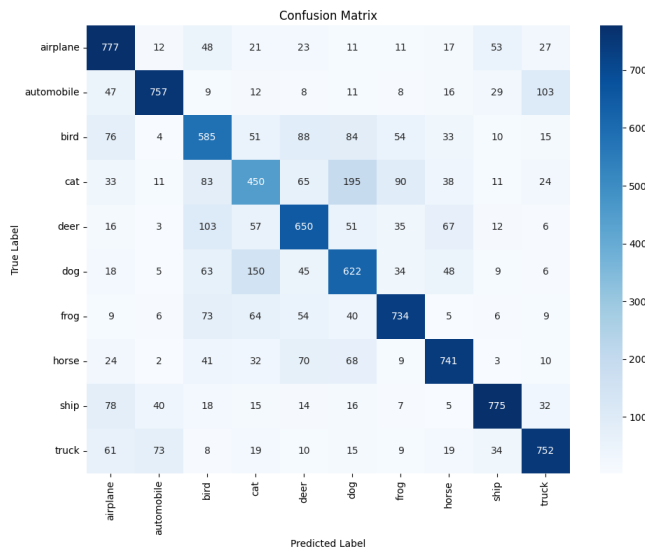
(a) Baseline CNN: Training accuracy (blue) climbs to $\sim 97\%$ while validation accuracy plateaus at $\sim 68\%$. This severe gap indicates overfitting without regularization.



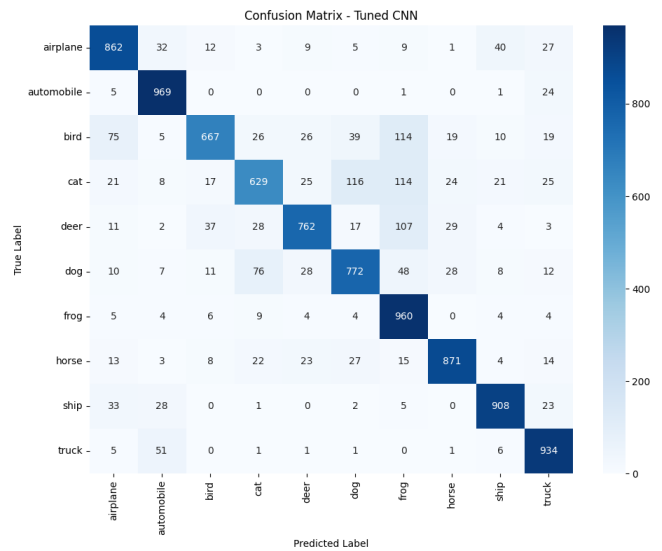
(b) Optimized CNN: Training and validation curves stay much closer together ($\sim 83\%$ val), demonstrating effective regularization via BN + Dropout + L2 + Augmentation.

Figure 7: Training vs Validation accuracy and loss for the Baseline CNN (left) vs Optimized CNN (right). The baseline exhibits classic overfitting: training loss $\rightarrow 0$ while validation loss *increases*. The optimized model's regularization strategies keep the gap narrow.

12.4 Confusion Matrix Comparison



(a) Baseline CNN confusion matrix. Cat (450/1000), Dog (622/1000), Automobile (757/1000) show significant misclassifications.



(b) Optimized CNN confusion matrix. Automobile (969/1000), Frog (960/1000), Truck (934/1000) are classified with high accuracy. Cat (629/1000) remains the hardest class.

Figure 8: Side-by-side confusion matrices. The diagonal values in the optimized model are consistently higher, confirming improved per-class recall across all 10 categories.

13 Results Summary

Metric	Baseline CNN	Optimized CNN
Training Accuracy (final)	~97%	~83% (train $\approx$ val)
Testing Accuracy	68.43%	83.34%
Precision	68.64%	83.85%
Recall	68.43%	83.34%
F1-score	68.44%	83.07%

Table 5: Final results summary for both models on the CIFAR-10 test set.

14 Discussion

1. Why does the optimized model generalize better?

The combination of Batch Normalization (stable training), Dropout (prevents co-adaptation), L2 regularization (weight decay), and data augmentation (effective dataset expansion) collectively reduce overfitting. The baseline model memorizes training data; the optimized model learns generalizable features.

2. Role of GlobalAveragePooling2D:

GAP replaces a large Flatten + Dense block (which in the 128-filter case would produce $4 \times 4 \times 128 = 2048$ features connected to 256 neurons, adding $2048 \times 256 = 524,288$ parameters). GAP reduces this to $128 \times 256 = 32,768$ parameters, controlling model complexity while preserving spatial feature information.

3. Why ReLU outperforms Sigmoid?

Sigmoid saturates when activations are large, producing near-zero gradients (vanishing gradient problem). ReLU's linear gradient for $x > 0$ allows efficient gradient propagation through deep networks. In 5 epochs, ReLU reached 57.05% vs Sigmoid's 33.29%.

4. Effect of ReduceLROnPlateau:

The LR was halved ($0.001 \rightarrow 0.0005$) at epoch 6 when validation loss stalled. This allowed the optimizer to take finer steps, leading to a jump from 54.43% to 66.37% validation accuracy at epoch 7, and further gains to 83%+.

5. Why Cat and Dog remain difficult?

Cat and Dog share similar textures, fur patterns, and poses. The model confuses Cat $\leftrightarrow$ Dog (116+114 errors) and Cat $\leftrightarrow$ Frog (114 errors). Techniques like attention mechanisms or data-class specific augmentation could reduce this confusion further.

6. Significance of Early Stopping:

EarlyStopping (patience=10) halted training once validation loss failed to improve for 10 consecutive epochs, restoring the best-weight checkpoint. This prevents wasted computation and avoids divergence in later epochs.

15 Additional Exercises

1. Output size for 64×64 image, 5×5 kernel, stride 2, padding 2:

$$\frac{64 - 5 + 2 \times 2}{2} + 1 = \frac{63}{2} + 1 = 32 \times 32$$

Keras-verified output shape: (1, 32, 32, 8).

2. Trainable parameters for 64 filters of size 3×3 , RGB input:

$$(3 \times 3 \times 3 + 1) \times 64 = 1,792 \text{ parameters}$$

3. ReLU vs Sigmoid – comparison:

See Section 6 and Figure 3. ReLU achieves 57.05% vs Sigmoid’s 33.29% in 5 epochs on CIFAR-10 due to better gradient flow.

4. Max Pooling vs Average Pooling:

Both pooling types were compared using a 2-Conv baseline model for 3 epochs. Max Pooling consistently outperforms Average Pooling in validation accuracy on CIFAR-10 because max-pooled features preserve the most prominent activations (dominant edges, textures) rather than averaging them out.

5. Effect of increasing filters from 16 to 64:

Increasing filters allows the network to learn a richer set of features. However, it also increases the parameter count and computation. The optimized model uses $32 \rightarrow 64 \rightarrow 128$ filters with GAP to balance expressiveness and overfitting risk via progressive regularization.

16 References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
2. Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML*.
3. Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*.
4. He, K., et al. (2016). Deep Residual Learning for Image Recognition. *CVPR*.
5. Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
6. TensorFlow/Keras Documentation: https://www.tensorflow.org/api_docs/python/tf/keras
7. CIFAR-10 Dataset: <https://www.cs.toronto.edu/~kriz/cifar.html>